
Dubai Final Process Documentation

Overall process

The complete Dubai data process runs in four main stages.

Stage 1:

Create DB1 transaction output separately.

Stage 2:

Create DB2 project output separately.

Stage 3:

After DB1 and DB2 are created, run the latitude and longitude enrichment process. This process reads both DB files, extracts unique project/location combinations, fetches coordinates using geopy, and writes the coordinates back into both DB files.

Stage 4:

Run the checklist process. The checklist creates an Excel analysis report and validates DB1 or DB2 against the respective schema file.

Folder structure

DB1 pipeline folder:

D:\Dubai Data\Dubai_DB1_pipe_line

DB2 pipeline folder:

D:\Dubai Data\Dubai_DB2

Latitude/longitude pipeline folder:

D:\Dubai Data\Lan_lont_PipeLine

Checklist folder:

D:\Dubai Data\CheckList

Main final documentation:

D:\Dubai Data\Dubai_Final_Process_Documentation.pdf

Stage 1: DB1 creation

Script:

D:\Dubai Data\Dubai_DB1_pipe_line\dubai_db1_pipe_line.py

Purpose:

Creates DB1 transaction-level data from raw Dubai transaction records.

Main input:

D:\Dubai\Dubai_Updated_data\Raw_Data\Transactions.csv

Main output:

Dubai_DB1 output CSV file.

Important output columns for latitude/longitude enrichment:

index

project_name
location_name
project_latitude
project_longitude
location_latitude
location_longitude

Notes:

The DB1 script already creates the latitude and longitude columns. They are initially blank and are filled later by the latitude/longitude pipeline.

Stage 2: DB2 creation

Script:

D:\Dubai Data\Dubai_DB2\generate_db2_pipeline.py

Purpose:

Creates DB2 project-level data from Projects, Units, Buildings, and Developers source files.

Main inputs:

D:\Dubai\Dubai_Updated_data\Raw_Data\Projects.csv
D:\Dubai\Dubai_Updated_data\Project_Unit_Summary\Units_with_actual_area_sqft.csv
D:\Dubai\Dubai_Updated_data\Raw_Data\Buildings.csv
D:\Dubai\Dubai_Updated_data\Raw_Data\Developers.csv

Main output:

Dubai_DB2 output Excel file.

Important output columns for latitude/longitude enrichment:

index
project_name
location_name
project_latitude
project_longitude
location_latitude
location_longitude

Notes:

The DB2 script also keeps latitude and longitude columns in the output. They are filled after DB2 is created by the latitude/longitude pipeline.

Stage 3: Latitude and longitude enrichment

Script:

D:\Dubai Data\Lan_lont_PipeLine\lat_long.py

Purpose:

Fetch project and location coordinates for unique project/location records found in both DB1 and DB2, then assign those coordinates back into both DBs.

Default DB files:

DB1 = D:\Dubai\Dubai_DB1_test.csv
DB2 = D:\Dubai\Dubai_DB2_test.xlsx

Customizable paths:

The paths are defined at the top of lat_long.py:

```
DB1_PATH = Path(r"D:\Dubai\Dubai_DB1_test.csv")
DB2_PATH = Path(r"D:\Dubai\Dubai_DB2_test.xlsx")
```

To run the same logic on another file, change these two path values or call:
`run_lat_long_pipeline(db1_path, db2_path)`

Step 3.1: Read DB1 and DB2

DB1 is read as CSV.
DB2 is read as Excel.

The pipeline expects both files to contain:

- index
- project_name
- location_name

The pipeline also expects or creates these output columns:

- project_latitude
- project_longitude
- location_latitude
- location_longitude

Step 3.2: Fetch unique project/location records

The pipeline selects these columns from both DB files:

- index
- project_name
- location_name

It combines DB1 and DB2 records into one dataframe.

First uniqueness step:

Duplicate rows are removed using index, project_name, and location_name.

Second uniqueness step:

Duplicate records are removed using index only, because index is treated as the unique project key.

Temporary unique file:

D:\Dubai Data\Lan_lont_PipeLine\temp_unique_projects.csv

This file contains the unique index, project_name, and location_name values that will be used for coordinate fetching.

Step 3.3: Fetch latitude and longitude using geopy

Library used:

geopy

Geocoder used:

Nominatim

Rate limiting:

The script uses geopy RateLimiter to avoid sending requests too quickly.

Project coordinate search:

The project search includes both project_name and location_name because this improves the chance of finding the correct Dubai project.

Example project search patterns:

- project_name, location_name, Dubai, United Arab Emirates
- project_name location_name, Dubai, UAE
- project_name, Dubai, United Arab Emirates

Location coordinate search:

The location search uses location_name with Dubai/UAE context.

Example location search patterns:

- location_name, Dubai, United Arab Emirates
- location_name, Dubai, UAE

Coordinate result file:

D:\Dubai Data\Lan_Iont_PipeLine\coordinates_results.csv

This file stores:

- index
- index_key
- project_name
- location_name
- project_latitude
- project_longitude
- location_latitude
- location_longitude
- project_search_query
- location_search_query

Resume behavior:

If coordinates_results.csv already exists, the script skips indexes already present in that file and only fetches missing coordinates.

Step 3.4: Assign coordinates back to DB1 and DB2

The script assigns coordinates back to both files using normalized index matching.

Columns updated in DB1:

- project_latitude
- project_longitude
- location_latitude
- location_longitude

Columns updated in DB2:

- project_latitude
- project_longitude
- location_latitude
- location_longitude

Backup files:

Before overwriting the original DB files, the script creates backup files:

- Dubai_DB1_test_before_lat_long.csv
- Dubai_DB2_test_before_lat_long.xlsx

Updated files:

- D:\Dubai\Dubai_DB1_test.csv
- D:\Dubai\Dubai_DB2_test.xlsx

How to run latitude/longitude pipeline

Command:

```
python D:\Dubai Data\Lan_lont_PipeLine\lat_long.py
```

Alternative from Python:

```
from Lan_lont_PipeLine.lat_long import run_lat_long_pipeline

run_lat_long_pipeline(
    db1_path=r"D:\Dubai\Dubai_DB1_test.csv",
    db2_path=r"D:\Dubai\Dubai_DB2_test.xlsx"
)
```

Important geopy note

Nominatim is a public geocoding service. It should be used with a clear user agent and rate limiting. For very large files, coordinate fetching can take time because the script intentionally slows down requests.

Stage 4: Checklist validation

Script:

```
D:\Dubai Data\CheckList\db_checklist.py
```

Purpose:

Creates a report for the generated DB file and validates the uploaded file against the matching schema.

DB1 schema:

```
D:\Dubai Data\CheckList\DB1_Schema.xlsx
```

DB2 schema:

```
D:\Dubai Data\CheckList\DB2_Schema.xlsx
```

DB1 validation columns:

```
DB1(Transaction DB)-Actual
Data_Type
```

DB2 validation columns:

```
DB2(Project_DB)-Actual
Data_Type
```

Automatic schema detection:

If the uploaded filename contains DB1, DB1_Schema.xlsx is used.
If the uploaded filename contains DB2, DB2_Schema.xlsx is used.

Checklist report sheets:

```
Column Analysis
Summary
First 10 Rows Sample
Schema Validation
```

Recommended final run order

1. Run DB1 pipeline:

```
python D:\Dubai Data\Dubai_DB1_pipe_line\dubai_db1_pipe_line.py
```

2. Run DB2 pipeline:
`python D:\Dubai Data\Dubai_DB2\generate_db2_pipeline.py`
3. Run latitude/longitude enrichment:
`python D:\Dubai Data\Lan_lont_PipeLine\lat_long.py`
4. Run checklist validation:
`python D:\Dubai Data\CheckList\db_checklist.py`

Final expected outcome

After all stages are complete:

- DB1 contains transaction data with project/location coordinates filled.
- DB2 contains project data with project/location coordinates filled.
- A checklist Excel report validates column completeness and data types.
- Documentation PDFs are available for DB1, DB2, checklist, and the full final process.